```
mov x1, x2
mov x2, x3
mov x3, x4
mov x4, x5
mov x5, x6
svc #0          // 陷入 EL1
ret             // 返回值已在 x0
```

user/libc_syscall.c 里的公开 libc 函数（fork、read、write、exec……）都是对这个网关的薄包装：一个 sc() 帮助函数把 SYS_* 号作为第一个实参传给 __syscall，再用 SC0/SC1/…/SC6 按参数个数展开（I/P 只做到 long 的转换）：

```
extern long __syscall(long, long, long, long, long, long, long);

static long sc(long number, long a0, long a1, long a2, long a3, long a4, long a5){
  long result = __syscall(number, a0, a1, a2, a3, a4, a5);
  if(result < 0)                  // 内核目前只返回 -1，尚无 Linux 式 -errno
    errno = EIO;
  return result;
}

#define SC3(n,a,b,c) sc(SYS_##n, (a), (b), (c), 0, 0, 0)
int write(int fd, const void *buf, int count)
{ return SC3(write, I(fd), P(buf), I(count)); }
```

这样分层有两个好处。其一，svc 只出现在这一个网关里，应用和大部分 libc 都不再直接执行陷入指令，系统调用号这套内核 ABI 被收进 libc.so 内部。其二，在动态链接下（第 8.5.2 节），应用对 write 的调用经 PLT 落到共享 libc，再由 libc 内部经 __syscall 陷入，用户程序镜像里不再各自携带一份桩代码。

内核侧 syscall() 用 trapframe->x7 索引函数表，返回值写回 trapframe->x0。目前共有 70 多个系统调用：除了 xv6 原有的 21 个，还有同步对象、网络（UDP、ICMP、TCP socket、epoll）、挂载、TTY 会话、线程、调度属性、高精度时钟等。

笔记 系统调用号放在 x7，这是本项目沿用的约定；Linux AArch64 用的是 x8。所以 Linux 的二进制或汇编系统调用包装不能直接拿来用，第 8.5 节的 POSIX 层是源码兼容，不是二进制兼容。

8.1.4 内核如何分发系统调用

svc 进入 el0sync，usavereg 把全部用户寄存器存进 trapframe，usertrap() 检查 ESR_EL1 的异常类别是否为 0x15（来自 AArch64 的 SVC），然后打开中断、调用 syscall()（第 3.2 节）。分发函数在 kernel/syscall.c 中：

```
void syscall(void)
{
  int num;
  struct proc *p = myproc();

  num = p->trapframe->x7;
  if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    p->trapframe->x0 = syscalls[num]();
  } else {
    printf("%d %s: unknown sys call %d\n", p->pid, p->name, num);
```